



조건 및 분기

예상 소요 시간: 10분

목표:

이번 읽기에서는 다음에 대해 배울 것입니다:

1. 비교 연산자
2. 분기
3. 논리 연산자

1. 비교 연산

비교 연산은 프로그래밍에서 필수적입니다. 이들은 값을 비교하고 결과에 따라 결정을 내리는 데 도움을 줍니다.

동등 연산자

동등 연산자 `==`는 두 값이 같은지 확인합니다. 예를 들어, Python에서:

```
age = 25
if age == 25:
    print("You are 25 years old.")
```

여기서 코드는 변수 `age`가 25와 같은지 확인하고 그에 따라 메시지를 출력합니다.

부등호 연산자

부등호 연산자 `!=`는 두 값이 같지 않은지 확인합니다:

```
if age != 30:
    print("You are not 30 years old.")
```

여기서 코드는 변수 `age`가 30이 아닌지 확인하고 그에 따라 메시지를 출력합니다.

크고 작음

하나의 값이 다른 값보다 큰지 비교할 수도 있습니다.

```
if age >= 20:
    Print("Yes, the Age is greater than 20")
```

여기서 코드는 변수 `age`가 20 이상인지 확인하고 그에 따라 메시지를 출력합니다.

2. 분기

분기는 프로그램에서 조건에 따라 결정을 내리는 것과 같습니다. 실제 생활의 선택으로 생각해 보세요.

IF 문

바에 들어가는 실제 상황을 고려해 보세요. 특정 나이를 초과하면 들어갈 수 있지만, 그렇지 않으면 들어갈 수 없습니다.

```
age = 20
if age >= 21:
    print("You can enter the bar.")
else:
    print("Sorry, you cannot enter.")
```

여기서 if 문을 사용하여 나이 변수에 따라 결정을 내리고 있습니다.

ELIF 문

때때로 여러 조건을 확인해야 할 때가 있습니다. 예를 들어, 바에 갈 나이가 충분하지 않다면 대신 영화에 갈 수 있습니다.

```
if age >= 21:
    print("You can enter the bar.")
elif age >= 18:
    print("You can watch a movie.")
else:
    print("Sorry, you cannot do either.")
```

실제 사례: 자동화 현금 인출기 (ATM)

사용자가 ATM과 상호작용할 때, ATM의 소프트웨어는 사용자의 입력에 따라 결정을 내리기 위해 분기를 사용할 수 있습니다. 예를 들어, 사용자가 “현금 인출”을 선택하면, ATM은 요청된 금액에 따라 다양한 지폐 단위로 분기하여 지급할 수 있습니다.

```
user_choice = "Withdraw Cash"
if user_choice == "Withdraw Cash":
    amount = input("Enter the amount to withdraw: ")
    if amount % 10 == 0:
        dispense_cash(amount)
    else:
        print("Please enter a multiple of 10.")
else:
    print("Thank you for using the ATM.")
```

3. 논리 연산자

논리 연산자는 조건을 결합하고 조작하는 데 도움을 줍니다.

NOT 연산자

실제 사례: 알림 설정

스마트폰의 알림 설정에서는 NOT 연산자를 사용하여 알림을 보낼 시점을 제어할 수 있습니다. 예를 들어, “방해 금지” 모드가 아닐 때만 알림을 받고 싶을 수 있습니다.

not 연산자는 조건을 부정합니다.

```
is_do_not_disturb = True
if not is_do_not_disturb:
    send_notification("New message received")
```

AND 연산자

실제 사례: 접근 제어

보안 시설에서는 AND 연산자를 사용하여 접근을 위한 여러 조건을 확인할 수 있습니다. 고급 보안 문을 열기 위해서는 유효한 ID 카드와 일치하는 지문이 모두 필요할 수 있습니다.

AND 연산자는 모든 필수 조건이 참인지 확인하며, 이는 금고를 열기 위해 두 개의 열쇠가 모두 필요한 것과 같습니다.

```
has_valid_id_card = True
has_matching_fingerprint = True
if has_valid_id_card and has_matching_fingerprint:
    open_high_security_door()
```

OR 연산자

실제 사례: 영화 밤 결정

친구들과 영화 밤을 계획할 때, OR 연산자를 사용하여 영화 장르를 결정할 수 있습니다. 최소한 한 사람이 관심이 있다면 영화를 선택할 것입니다.

OR 연산자는 최소한 하나의 조건이 참인지 확인합니다. 여러 영화를 선택하는 것과 같습니다.

```
friend1_likes_comedy = True
friend2_likes_action = False
friend3_likes_drama = False
if friend1_likes_comedy or friend2_likes_action or friend3_likes_drama:
    choose a movie()
```

요약

이 읽기에서는 가장 자주 사용되는 연산자와 if 문 및 if-else 문을 포함하는 조건 분기 개념에 대해 다루었습니다.

저자

[아칸샤 야다브](#)



Skills Network

변경 로그

날짜	버전	변경자	변경 설명
2023-11-03	1.0	Akansha Yadav	읽기 파일 생성
2024-02-19	2.0	Gagandeep	ID 검토
->			